Projet LOA 2025-2026 CHEN-HUYNH
BABA IS YOU

Ce qui a été traité

Le jeu de base, où on peut se déplacer sur la map, avec les règles de base, BABA, WALL, ROCK, FLAG avec les propriétés YOU, STOP, PUSH, WIN et l'opérateur IS.
On a aussi traité le undo/redo, utilisé le pattern Modèle / Vue / Contrôleur et redéfinit l'opérateur [] pour pouvoir utiliser board[position] afin d'avoir les entités à cette position.
Les extensions réalisées: PULL, KEY, DOOR, WALL IS BABA, AND et suite de niveaux.
Une expérience utilisateur agréable, simple, ergonomique, intuitive, fluide et complète.

Aspects les plus significatifs

Jeu de base: une map qui est représentée par un vector<vector<Cell>>, Cell pour permettre d'avoir plusieurs éléments dans une case.

Règle: la gestion des règles est gérée en temps réel, selon les mouvements du joueur et les règles actives. Permettant ainsi d'avoir des changements de propriété ou d'entité.

PULL: il est possible de PULL les textes sur la map et pour les entités il est nécessaire d'avoir une règle NOUN IS PULL.

KEY et DOOR: l'ajout des mots clés SHUT et OPEN ont été primordiale afin de pouvoir avoir cette règle de manière complète, le OPEN permet d'ouvrir le SHUT.

WALL IS BABA: on a dû changer la structure des règles permettant NOUN IS PROPERTY/NOUN, et ensuite si on a une propriété dans ce cas on exécute la propriété, sinon si on a un noun, on transforme tous les NOUN (droite) par NOUN (gauche).

AND, il suffit de regarder après le IS NOUN/PROPERTY si AND est présent ou si on a un format NOUN AND NOUN … IS. Si oui, dans ce cas on fait une distribution sur les règles, et on continue de regarder en arrière jusqu'à qu'on ait plus de AND.

Pattern MVC: utilisation du MVC pour séparer la logique et l'affichage lié par le contrôleur permettant une maintenance facile et compréhensible.

Pattern Singleton: avoir une unique et seule instance pour éviter tout problème de conflits ou de bugs et avoir un jeu optimisé.

Pattern Iterator: pour parcourir les vectors sans donner d'information sur la manière dont elles sont stockées.

Pattern Memento: se souvient des états des entités permettant l'utilisation de l'undo/redo.

Pattern Factory: la création des objets Entity à partir du JSON est séparée dans un coin spécifique du code, ce qui permet de créer des objets facilement sans polluer le code.

Pattern Template Method : permet aux sous classes de redéfinir certaines méthodes sans changer la structure des Entity.

Problème connus

On a pas vraiment de problème en terme de gameplay, mais sinon sur l'affichage, on a un petit problème, dans le cas ou on a par exemple un ROCK et BABA sur une case, ça risque d'afficher soit que le rocher soit que le baba et pas les deux en même temps contrairement au jeu original.

Etats des extensions qu'on a pas implémenter

On a pas implémenter une seule extension, qui est celle de jouer sur le contrôle (A IS LEFT).

- A IS LEFT, on a compris tardivement cette extension, ce qui nous a empêchés de le coder, mais dans l'idée, ça serait de regarder si on a des règles dans le genre TOUCHE IS DIRECTION sur la map. Si oui, on fait en sorte que le controller keyboardListener regarde les règles d'abord et si on a une règle TOUCHE IS DIRECTION, dans ce cas on exécute la direction.Sinon, on garde les touches par défaut.

Diagrammes UML

